KernelScript: Compiler-Checked Cross-Boundary Typed DSL for eBPF Applications

Cong Wang[1], Siyuan Sun[2], Yusheng Zheng[3]
[1]Multikernel Technologies [2]Xi'an University of Posts and Telecommunications [3]UC Santa Cruz & Eunomia Labs

Abstract

eBPF has become a standard mechanism for extending Linux with custom packet processing, tracing, and scheduling logic, with a verifier check before execution to guarantee it will not crash the kernel. The programming model, however, is fragmented. A single application spans kernel code, a userspace loader, and shared maps, yet the relationships among them (types, lifecycle ordering, shared state) go unchecked. An event struct defined differently on each side silently garbles every record, and a map type mismatch silently corrupts shared state. We observe that these cross-boundary relationships are duplicated concepts that a type system can unify. We present KernelScript, a DSL that types maps, program handles, and execution domains in one source, then lowers to standard C through the clang/bpftool/libbpf toolchain. The compiler rejects mismatches such as wrong context types and map-schema disagreements before code generation, including published verifier-accepted exemplars that stock C/libbpf still loads. A unified source also reduces cross-file change amplification, and generated projects load, attach, and pass traffic on a real kernel while preserving the existing toolchain.

CCS Concepts: • **Software and its engineering → Domain specific languages**; *Static analysis*; • **Computer systems organization →** *Reliability*.

Keywords: eBPF, domain-specific language, static typing, systems

1 Introduction

eBPF lets developers run application-specific logic inside the Linux kernel under a verifier that proves memory safety and bounded execution [21]. It now underpins packet processing in major cloud providers, powers observability platforms, and enables scheduler experimentation within a large and growing ecosystem [23].

Yet the programming model is fragmented. A single eBPF application spans kernel code constrained by the verifier, a userspace loader that opens objects and manages lifecycle, shared maps and ring buffers, and (for newer features) generated skeleton headers that must match the running kernel. The relationships among these pieces, such as a map's key and value types or the order of load, attach, and detach, are checked only at load time or not at all. Errors may go entirely undetected because they do not break the kernel. When a map's value struct is redefined in kernel C but not in the userspace loader, data is silently misinterpreted at runtime. When a skeleton header becomes stale after a struct change, userspace compiles against the old layout and reads corrupt fields.

Existing tools reduce boilerplate yet leave cross-boundary relationships implicit. The standard eBPF development workflow requires hand-written kernel C, userspace C, and a Makefile [15]. Aya compiles both sides from Rust and shares typed maps [1], and cilium/ebpf does similarly in Go [3]. In both, lifecycle ordering remains a runtime property. bpftrace targets tracing with higher-level abstractions [2] rather than general eBPF applications. No existing tool types the full cross-boundary structure at compile time.

The key observation is that these relationships are single concepts duplicated across components. A map declaration is simultaneously a kernel data structure, a userspace lookup target, and a type contract. A program handle is simultaneously a function name, a BPF section, and a file descriptor. In C/libbpf, naming conventions connect these copies, so mismatches surface late. In a unified typed source, the compiler can see and check them directly.

KernelScript realizes this idea as a compiled, multi-domain DSL. A developer writes eBPF entry points, userspace control flow, maps, and ring buffers in one source file. The compiler types maps, program handles, and execution domains, then lowers to eBPF C, userspace C, and a Makefile. The standard libbpf and bpftool path then produces the final binaries, preserving the existing toolchain.

This paper contributes:

- A *typed model of cross-boundary eBPF structure*: execution domains, typed maps, and program-lifecycle handles, each with a typing rule and matching compiler diagnostic (Section 3).
- A compiler that realizes this model while preserving the stock kernel toolchain, so compatibility failures remain visible (Section 4).
- An evaluation of early detection on published verifier-accepted defects, reduced change amplification versus C/libbpf, and stock-toolchain compatibility on a real kernel (Section 5).

2 Background and Motivation

eBPF allows user-supplied bytecode to run inside the Linux kernel after a verifier proves memory safety and bounded execution [21, 23]. The mechanism now underpins a wide range of applications: tracing and profiling [2, 12], high-performance networking [9], LSM policy enforcement [14], CPU scheduling [11], page-cache customization [30], GPU resource control [28], and userspace extension runtimes [27]. Every eBPF application has a kernel program and a userspace loader that communicate through maps or ring buffers. The standard eBPF development workflow requires developers to maintain separate kernel C, userspace C, and a Makefile [15]. Aya unifies both sides in Rust [1], and Rex replaces the verifier with language-level safety in Rust [13], but neither types the full cross-boundary structure at compile time.

The kernel/userspace split introduces three kinds of coupling that current tools do not fully address. *Lifecycle coupling* requires operations to occur in a valid order (load before attach, detach before close), but C/libbpf checks this only at runtime. *Representation coupling* requires a map's key and value types to agree across kernel code, generated headers, and userspace lookups, yet a mismatch is silent until something fails. *Compatibility coupling* arises because generated bindings depend on the exact kernel and libbpf versions. The verifier guarantees memory safety but not application correctness across the boundary: calling `attach` before populating maps lets the kernel program run against empty state. Redefining a ring-buffer event struct on one side without updating the other silently garbles every record. A stale skeleton header after a kernel struct change compiles without warning but misaligns every shared field. Automated eBPF-to-Rust migration confirms that these implicit cross-boundary relationships are a primary source of translation errors [4]. A DSL can move lifecycle and representation checks to compile time.

3 A Typed Model of Cross-Boundary Structure

Every rule corresponds to a diagnostic exercised by the rejection suite in Section 5.

A KernelScript program is a flat sequence of declarations. Listing 1 shows the minimal shape of an application. The rest of this section makes its three relationships precise.

Listing 1. A unified-source eBPF application.

```
include "xdp.kh"

@xdp fn pass_all(ctx: *xdp_md) -> xdp_action {
  return XDP_PASS
}

fn main() -> i32 {
  var prog = load(pass_all)
  attach(prog, "lo", 0)
  detach(prog)
  return 0
}
```

To address representation coupling, each function carries an attribute that assigns it to an execution domain $d \in \{\mathit{user}, \mathit{ebpf}, \mathit{helper}, \mathit{kfunc}\}$. The attributes `@xdp`, `@tc`, `@probe`, `@tracepoint`, and `@perf_event` mark eBPF entry points, while `@helper` marks kernel-shared eBPF code and `@kfunc` marks kernel-module code. An unattributed `fn` is userspace. The checker tags each function with its domain and rejects illegal crossings, such as a userspace function calling an `@helper`. Each eBPF attribute also fixes a context and return type: for example, `@xdp` requires `ctx:*xdp_md` returning `xdp_action`, and `@tc` requires `*__sk_buff` returning `i32`. The compiler rejects any signature that disagrees with its attribute before code generation. The same surface syntax thus remains domain-aware: userspace can allocate and print freely, while eBPF code must obey stack and helper restrictions. struct_ops callbacks fit the same scheme, with context and return types determined by the registered slot.

To address representation coupling for shared state, a map is declared as a typed global (`var m : hash<K,V>(N)`) rather than a C section plus separate userspace lookup code. The compiler checks element types at every access site on both sides of the kernel boundary: an index `m[k]` requires `k:K` and yields `V`, in eBPF and userspace alike. A wrong key type, wrong value type, or access to an undeclared map is a compile-time error. One declaration thus replaces kernel definitions, generated fields, and userspace lookups that otherwise must agree by hand.

To address lifecycle coupling, the type system distinguishes a `FunctionRef` (a reference to an `@`-attributed function) from a `ProgramHandle` (a loaded program). The typing ensures that only `load` produces a `ProgramHandle`. The formation rule for `FunctionRef` connects lifecycle to domains: only an eBPF entry-point function inhabits `FunctionRef`, so an `@helper`, `@kfunc`, or userspace function cannot be passed to `load`.

$$\frac{f \text{ has an eBPF entry attribute}}{\Gamma \vdash f : \texttt{FunctionRef}}$$

$$\frac{\Gamma \vdash f : \texttt{FunctionRef}}{\Gamma \vdash \texttt{load}(f) : \texttt{ProgramHandle}}$$

$$\frac{\Gamma \vdash h : \texttt{ProgramHandle}}{\Gamma \vdash \texttt{detach}(h) : \texttt{unit}}$$

$$\frac{\Gamma \vdash h : \texttt{ProgramHandle}}{\Gamma \vdash \texttt{attach}(h, t, fl) : \texttt{u32}}$$

Because `attach` and `detach` require a `ProgramHandle` and only `load` produces one, "attach before load" is a *type* error, not a runtime failure. Passing a bare `FunctionRef`, an integer, or a string to a lifecycle builtin is a type error. The typing rules enforce a producer/consumer handle discipline. Because only `load` produces a `ProgramHandle`, the system enforces the single ordering constraint $\mathit{load} \prec \{\mathit{attach}, \mathit{detach}\}$ rather than the complete load-to-attach-to-detach protocol. It guarantees that lifecycle builtins receive values of the right kind. Full path-sensitive properties such as use-after-detach,

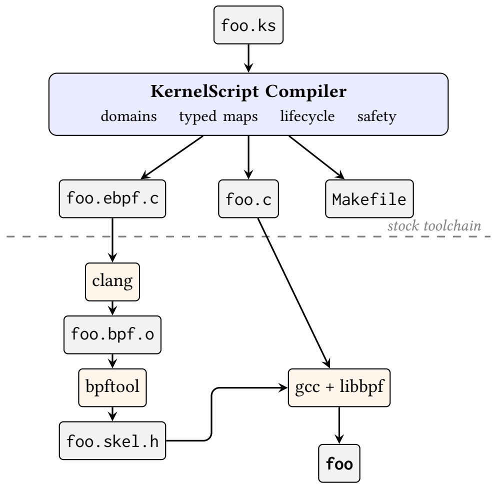

Figure 1. Compilation pipeline. The compiler checks cross-boundary invariants and emits C; the stock clang/bpftool/libbpf toolchain produces the final binary.

double-attach, or leaked handles remain outside the type system's scope.

The benefit is localized compiler errors earlier in development, replacing naming conventions and late verifier or attach-time failures. Section 5 measures early detection against published defects, change amplification, and stock-toolchain compatibility. Section 6 states the limits of that evidence.

4 Design and Implementation

The compiler has three design goals. First, it must detect cross-boundary errors at compile time, before the verifier or runtime sees them. Second, it must preserve the existing Linux BPF toolchain so that type information, portability, and verifier diagnostics remain available. Third, it must generate idiomatic C that developers can inspect and debug.

The type system draws on established ideas. Typestate tracks object state in types [20], and linear types enforce single-use resource protocols [5, 10, 24]. KernelScript applies a lightweight form of these ideas, distinguishing producer and consumer handles and annotating execution domains without full substructural typing.

To meet these goals, the compiler (written in OCaml with dune) parses KernelScript source, resolves imports and includes, builds symbol tables, runs multi-program analysis and type checking, performs safety analysis, builds an IR, and emits C (Figure 1). The compiler comprises nine modules covering maps, codegen, safety, and struct_ops. For an input `foo.ks`, the generated project contains `foo.ebpf.c`, `foo.c`, a `Makefile`, and, when kfuncs are present, `foo.mod.c`. The Makefile invokes bpftool to dump `vmlinux.h`, clang to produce a BPF object, bpftool to generate a skeleton, and gcc to link the userspace loader against libbpf (together with libelf and zlib).

Each typing rule of Section 3 maps to a concrete compiler pass. The compiler attaches a domain to each function as a scope refined by the entry attribute. The IR records each function's program type so that code generation can dispatch the same source function to the eBPF or userspace backend. Typed maps lower to a single descriptor that records key and value types once. Both backends read that descriptor to emit the eBPF `SEC(".maps")` definition and the userspace accessor (Listing 2). Lifecycle handles are enforced in the front end. Only `load` yields a `ProgramHandle`, and `attach` and `detach` require one, so a misordered call fails type checking before code generation. The handle then lowers to a file descriptor via libbpf APIs.

Listing 2. One typed map declaration lowers to both a kernel `SEC(".maps")` definition and a userspace accessor from a single descriptor.

```
// KernelScript source
var counts : hash<u32, u64>(1024)

// generated eBPF C
struct {
  __uint(type, BPF_MAP_TYPE_HASH);
  __uint(max_entries, 1024);
  __type(key, __u32);
  __type(value, __u64);
} counts SEC(".maps");

// generated userspace C: same key/value types
__u32 key = ...; __u64 val;
bpf_map_lookup_elem(counts_fd, &key, &val);
```

The compiler does not replace the verifier, clang, bpftool, or libbpf. Existing diagnostics therefore remain available, and any compatibility failure stays visible. For interfaces that move faster than the DSL can stabilize, `include` and `extern` declarations provide a fallback to raw header declarations.

5 Evaluation

The evaluation answers three questions. *(Q1)* Do typed cross-boundary checks reject independently defined, verifier-accepted defects earlier than stock C/libbpf? *(Q2)* Does unified typed source reduce cross-boundary change amplification versus hand-written C/libbpf? *(Q3)* Do generated objects remain compatible with the stock clang/bpftool/libbpf path on a real kernel? All experiments run on Linux 6.15 with clang 18, bpftool 7.7, and libbpf 1.3.0. SLOC counts exclude blank lines, comments, and generated headers.

5.1 Q1: Earlier Detection of Cross-Boundary Defects

A 28-program rejection suite (one positive control, 27 expected rejections with fixed diagnostic substrings) shows that each typing rule of Section 3 is wired: lifecycle misuses, signature violations, map-type errors, domain crossings, and selected safety checks fail before code generation. The unit suite (85 suites, 1095 tests) is supporting evidence that the compiler is intact, not a comparative claim.

The comparative question is whether KernelScript catches defects the stock path still accepts. We replay three published, verifier-accepted exemplars from Heimdall [4] (Listings 5–6): an XDP program typed with `struct __sk_buff`, a 16-byte update into an 8-byte map value, and a map lookup reinterpreted as an unrelated `u64`. For each defect we pair a buggy and corrected program in both KernelScript and C/libbpf, require an exact KernelScript diagnostic on the buggy source, and require the C object to clang-build, verifier-load, and pass a defect-specific `BPF_PROG_TEST_RUN` oracle over ten trials (map truncation to `{1,2}`, native-endian reinterpretation of `conn{1,2}` as `u64`, and successful field stores for the context pair).

Table 1. Published verifier-accepted exemplars (Heimdall). BS = KernelScript. All three are compile-time rejects in BS with the expected diagnostic; C/libbpf builds, loads, and executes the buggy objects.

Published defect	Family	BS	C/libbpf
XDP + `__sk_buff` context	signature/domain	compile reject	runtime accept
16B write into 8B map value	representation	compile reject	runtime trunc.
Map value reinterpreted as `u64`	representation	compile reject	runtime reinterpret

Table 1 shows KernelScript is strictly earlier on all three exemplars (two invariant families). Corrected siblings compile in KernelScript and pass the same C runtime harness, so rejections are not incidental parse failures. Local C mistakes such as undeclared maps or oversized stacks remain ties: clang already rejects them. Typing therefore helps on cross-boundary relations that byte-oriented C checking and the verifier do not treat as type errors. We do not claim prevalence beyond these exemplars, nor an advantage over Aya's typed Rust surface, which Heimdall already reports for the same classes.

5.2 Q2: Change Amplification

Across 11 matched workloads with hand-written C/libbpf baselines, KernelScript application sources total 203 lines versus 1105 for the full C/libbpf projects (median generated project size is 472 lines from 31 source lines). Hand-porting three `xdp-tutorial` [22] XDP programs yields line counts comparable to the original C/eBPF when porting existing code rather than writing from scratch. A source-only scan of three public eBPF repositories confirms that the program types and map patterns we generate appear together in real projects.

Source-footprint totals measure finished size but miss how many places a requirement change must touch. To measure change-amplification coupling, we construct five matched micro-edits (map `array`→`percpu_array`, program `@xdp`→`@tc`, perf_event standalone→grouped, shared ringbuf event schema, and adding ringbuf reporting with userspace handling) and compare diffs between KernelScript and hand-written C/libbpf. Table 2 reports file count, edit-site count, changed LOC, and required manual synchronization across kernel code, userspace code, or skeleton/section conventions.

Table 2. Change amplification for five matched micro-edits (BS = KernelScript, C = hand-written C/libbpf). "Sync" reports which manually synchronized surfaces change: kernel (K), userspace (U), or skeleton/section conventions (S).

Edit	Impl.	Files	Sites	LOC	Sync
Map `array`→`percpu_array`	BS	1	1	2	—
	C	2	7	19	K/U/S
Program `@xdp`→`@tc`	BS	1	2	8	—
	C	2	6	30	K/U/S
Perf event standalone→grouped	BS	1	1	6	—
	C	1	8	91	U
Shared event schema	BS	1	3	4	—
	C	2	4	7	K/U
Add ringbuf reporting/handling	BS	1	5	23	—
	C	2	11	89	K/U/S

None of the KernelScript edits require manual cross-boundary synchronization, whereas hand-written C/libbpf requires userspace synchronization in 5/5 cases, kernel-side synchronization in 4/5, and skeleton or section-convention synchronization in 3/5. Diff-size medians reinforce this: KernelScript touches 1 file, 2 edit sites, and 6 lines, versus 2, 7, and 30 for C/libbpf. With only five edits these numbers are illustrative, but they show that a unified typed source reduces change amplification beyond total line count. As cross-boundary structure accumulates (pass-only XDP → counter map → ring-buffer reporting), KernelScript grows more slowly: +2 lines for the counter map versus +11 in C/libbpf, and +12 for ring-buffer reporting versus +202. By the reporting stage, C/libbpf reaches 9× the KernelScript source size.

5.3 Q3: Stock-Toolchain Compatibility

Q3 asks whether typing preserves the stock path rather than replacing it. Of 43 repository examples spanning XDP, TC, kprobe, tracepoint, perf_event, maps, ring buffers, tail calls, kfuncs, and struct_ops, 41 build through the generated Makefile; the two failures are version-skew skeleton mismatches, which remain visible precisely because generation still uses bpftool/libbpf. On a real kernel, 27 single-section XDP objects attach and detach in isolated netns/veth pairs. Matched workloads for XDP/TC traffic, ring-buffer emission, perf_event counters, and selected struct_ops paths confirm functional equivalence under shared oracles. Under `BPF_PROG_TEST_RUN`, a minimal generated XDP pass matches its hand-written baseline in instruction count and latency; ten veth/iperf3 trials show comparable throughput. These are compatibility and equivalence checks, not performance claims.

6 Limitations

The early-detection comparison uses three published exemplars rather than bugs mined from production revision histories, so it does not establish prevalence. Context-field remapping is shown by loading and executing the wrong-typed XDP object; non-zero queue/ifindex injection via `BPF_PROG_TEST_RUN` ctx is unsupported on our host, so that case does not claim a non-zero field oracle. Lifecycle typing enforces a producer/consumer handle distinction without full typestate, so use-after-detach and leaked handles remain possible. Change-amplification uses five checked-in micro-edits, not developer-time studies. Traffic numbers are local sanity checks.

7 Related Work

Prior work on eBPF development falls into three categories. Domain-specific DSLs target particular use cases: bpftrace for tracing [2], BPFBox and ActPlane for policy enforcement [7, 26], BMC for caching [8], P4c-eBPF for networking [17], and Yaksha-Prashna for bytecode analysis [19]. These DSLs primarily target kernel-side code and do not type the full cross-boundary structure that KernelScript addresses. General eBPF development tools include the standard eBPF development workflow [15], Aya for Rust [1], Rex for safe Rust kernel extensions without a verifier [13], Wasm-bpf for WebAssembly deployment [29], and eunomia-bpf for dynamic loading [6]. These reduce boilerplate but leave lifecycle and representation relationships implicit. Verified approaches take a different direction: BeePL provides correct-by-compilation kernel extensions [18], and Jitk and Serval verify JIT correctness [16, 25]. KernelScript occupies a middle ground. It generates both kernel and userspace code from one source and types cross-boundary relationships. The goal is practical integration with the existing toolchain, not formal verification.

8 Conclusion

KernelScript treats the cross-boundary structure of an eBPF application (program lifecycle, shared maps, and execution domains) as something a compiler can type, replacing conventions and late verifier/attach-time discovery. On published verifier-accepted exemplars, typed checks reject hook/-context and map-schema defects before code generation while C/libbpf still builds and runs them; a unified source reduces multi-surface change amplification; and generated objects remain loadable under the stock toolchain on a real kernel. The result is a concrete design point: typing eBPF's cross-boundary structure is feasible without replacing clang, bpftool, or libbpf, so both the type system's guarantees and the version-skew limits it cannot absorb remain visible.

References

[1] Aya contributors. 2026. Aya: Rust eBPF library. https://github.com/aya-rs/aya.

[2] bpftrace developers. 2026. bpftrace: High-level tracing language for Linux eBPF. https://github.com/bpftrace/bpftrace.

[3] Cilium contributors. 2026. cilium/ebpf: eBPF library for Go. https://github.com/cilium/ebpf.

[4] Vishnu Asutosh Dasu, Monika Santra, Md Rafi Ur Rashid, Ashish Kumar, Saeid Tizpaz-Niari, and Gang Tan. 2026. Heimdall: Formally Verified Automated Migration of Legacy eBPF Programs to Rust. arXiv preprint arXiv:2605.25411.

[5] Robert DeLine and Manuel Fähndrich. 2001. Enforcing High-Level Protocols in Low-Level Software. In *Proceedings of the ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI)*. 59–69. doi:10.1145/378795.378811

[6] eunomia-bpf developers. 2024. eunomia-bpf: A Dynamic Loading Framework for eBPF. https://github.com/eunomia-bpf/eunomia-bpf.

[7] William Findlay, Anil Somayaji, and David Barrera. 2020. BPFBox: Simple Precise Process Confinement with eBPF. In *Proceedings of CCSW*. 91–103. doi:10.1145/3411495.3421358

[8] Yoann Ghigoff, Julien Sopena, Kahina Lazri, Antoine Blin, and Gilles Muller. 2021. BMC: Accelerating Memcached using Safe In-kernel Caching and Pre-stack Processing. In *Proceedings of the 18th USENIX Symposium on Networked Systems Design and Implementation (NSDI)*. 487–501. https://www.usenix.org/conference/nsdi21/presentation/ghigoff

[9] Toke Høiland-Jørgensen, Jesper Dangaard Brouer, Daniel Borkmann, John Fastabend, Tom Herbert, David Ahern, and David Miller. 2018. The eXpress Data Path: Fast Programmable Packet Processing in the Operating System Kernel. In *Proceedings of the 14th International Conference on Emerging Networking Experiments and Technologies (CoNEXT)*. 54–66.

[10] Kohei Honda, Vasco T. Vasconcelos, and Makoto Kubo. 1998. Language Primitives and Type Discipline for Structured Communication-Based Programming. In *Programming Languages and Systems (ESOP)*. 122–138. doi:10.1007/BFb0053567

[11] Jack Tigar Humphries, Neel Natu, Ashwin Chaugule, Ofir Weisse, Barret Rhoden, Josh Don, Luigi Rizzo, Oleg Noskov, Paul Turner, and Christos Kozyrakis. 2021. ghOSt: Fast & Flexible User-Space Delegation of Linux Scheduling. In *Proceedings of the 28th ACM Symposium on Operating Systems Principles (SOSP)*. 588–604. doi:10.1145/3477132.3483542

[12] iovisor/bcc developers. 2026. BCC: Tools for BPF-based Linux IO analysis, networking, monitoring, and more. https://github.com/iovisor/bcc.

[13] Jinghao Jia, Ruowen Qin, Milo Craun, Egor Lukiyanov, Ayush Bansal, Minh Phan, Michael V. Le, Hubertus Franke, Hani Jamjoom, Tianyin Xu, and Dan Williams. 2025. Rex: Closing the Language-Verifier Gap with Safe and Usable Kernel Extensions. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*. https://www.usenix.org/conference/atc25/presentation/jia

[14] KP Singh. 2020. LSM BPF: Kernel Runtime Security Instrumentation. https://docs.kernel.org/bpf/prog_lsm.html. Linux kernel 5.7+.

[15] libbpf developers. 2026. libbpf: a BPF CO-RE userspace library. https://github.com/libbpf/libbpf.

[16] Luke Nelson, James Bornholt, Ronghui Gu, Andrew Baumann, Emina Torlak, and Xi Wang. 2019. Scaling Symbolic Evaluation for Automated Verification of Systems Code with Serval. In *Proceedings of the 27th ACM Symposium on Operating Systems Principles (SOSP)*. 225–242. doi:10.1145/3341301.3359641

[17] P4 Language Consortium. 2024. P4c-eBPF: P4 Compiler Backend for eBPF. https://github.com/p4lang/p4c/tree/main/backends/ebpf.

[18] Swarn Priya, Frédéric Besson, Connor Sughrue, Tim Steenvoorden, Jamie Fulford, Freek Verbeek, and Binoy Ravindran. 2025. BeePL: Correct-by-Compilation Kernel Extensions. In *Principles of Secure Compilation (PriSC @ POPL)*. https://popl25.sigplan.org/details/prisc-2025-PriSC-2024-1/3/BeePL-Correct-by-compilation-kernel-extensions

[19] Animesh Singh, K. Shiv Kumar, S. VenkataKeerthy, Pragna Mamidipaka, R. V. B. R. N. Aaseesh, Sayandeep Sen, Palanivel A. Kodeswaran, Theophilus A. Benson, Ramakrishna Upadrasta, and Praveen Tammana. 2026. Yaksha-Prashna: Understanding eBPF Bytecode Network Function Behavior. arXiv preprint arXiv:2602.11232.

[20] Robert E. Strom and Shaula Yemini. 1986. Typestate: A Programming Language Concept for Enhancing Software Reliability. *IEEE Transactions on Software Engineering* SE-12, 1 (1986), 157–171. doi:10.1109/TSE.1986.6312929

[21] The Linux Kernel Documentation Project. 2026. eBPF Documentation. https://docs.kernel.org/bpf/.

[22] The XDP Project. 2026. xdp-tutorial: XDP Hands-On Tutorial. https://github.com/xdp-project/xdp-tutorial. Accessed 2026.

[23] Marcos A. M. Vieira, Matheus S. Castanho, Racyus D. G. Pacífico, Elerson R. S. Santos, Eduardo P. M. Câmara Juńior, and Luiz F. M. Vieira. 2021. Fast Packet Processing with eBPF and XDP: Concepts, Code, Challenges, and Applications. *Comput. Surveys* 53, 1 (2021), 1–36. doi:10.1145/3371038

[24] Philip Wadler. 1990. Linear Types Can Change the World! In *Programming Concepts and Methods*, M. Broy and C. Jones (Eds.). North-Holland. https://oa.mg/work/2912106379

[25] Xi Wang, David Lazar, Nickolai Zeldovich, Adam Chlipala, and Zachary Tatlock. 2014. Jitk: A Trustworthy In-Kernel Interpreter Infrastructure. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation (OSDI)*. 33–47. https://www.usenix.org/conference/osdi14/technical-sessions/presentation/wang_xi

[26] Yusheng Zheng, Tianyuan Wu, Quanzhi Fu, Tong Yu, Wenan Mao, Tao Ma, Dan Williams, Wei Wang, and Andi Quinn. 2026. ActPlane: Programmable OS-Level Policy Enforcement for Agent Harnesses. arXiv preprint arXiv:2606.25189.

[27] Yusheng Zheng, Tong Yu, Yiwei Yang, Yanpeng Hu, Xiaozheng Lai, Dan Williams, and Andi Quinn. 2025. Extending Applications Safely and Efficiently. In *19th USENIX Symposium on Operating Systems Design and Implementation (OSDI 25)*. https://www.usenix.org/conference/osdi25/presentation/zheng-yusheng

[28] Yusheng Zheng, Tong Yu, Yiwei Yang, Minghui Jiang, Xiangyu Gao, Jianchang Su, Yanpeng Hu, Wenan Mao, Wei Zhang, Dan Williams, and Andi Quinn. 2025. gpu_ext: Extensible OS Policies for GPUs via eBPF. arXiv preprint arXiv:2512.12615.

[29] Yusheng Zheng, Tong Yu, Yiwei Yang, and Andrew Quinn. 2024. Wasm-bpf: Streamlining eBPF Deployment in Cloud Environments with WebAssembly. arXiv preprint arXiv:2408.04856.

[30] Tal Zussman, Ioannis Zarkadas, Jeremy Carin, Andrew Cheng, Hubertus Franke, Jonas Pfefferle, and Asaf Cidon. 2025. cache_ext: Customizing the Page Cache with eBPF. In *Proceedings of the 30th ACM Symposium on Operating Systems Principles (SOSP)*. 462–478. doi:10.1145/3731569.3764820